EECE.4862/5862 Embedded Artificial Intelligence

Lab 1: Microcontroller Programming + First TFLite Micro Inference

last update: 5/20/26

General Information

Lab Mode: Remote. Work on the lab remotely using your own computer and lab dev kit.

Lab Due Date: Posted in Canvas. Submit all the required deliverables by the due date.

Lab Report Submission: Submit via Canvas. No email submissions are accepted.

Lab Contents: Additional requirements applied for EECE.5862. Details are below.

Lab Description

Requirements for both EECE.4862 and EECE.5862 students

This lab introduces the **Arduino Nano 33 BLE Sense Rev2** as the course's embedded AI target. You will set up the toolchain, exercise GPIO with a push-button and an LED on a breadboard, and then deploy a pre-trained **TensorFlow Lite Micro** (TFLM) model — the *hello-world* sine-wave demo — and characterize its on-device behavior.

The lab has two parts (A and B). Complete the following requirements:

(1) Install the Arduino IDE 2.x and the **Arduino Mbed OS Nano Boards** package. Plug the Nano 33 BLE Sense Rev2 into your laptop with a **data-capable** USB cable. In the IDE, set Tools → Board → *Arduino Nano 33 BLE* and Tools → Port → the port the Nano enumerates as. Open File → Examples → 01.Basics → **Blink**, compile, and upload. Confirm the on-board orange LED blinks at 1 Hz.

(2) Modify the Blink sketch to print a short message to Serial at every state change (e.g. `LED ON @ 1234 ms`) using `Serial.println()` and `millis()`. Open Tools → Serial Monitor at **9600 baud** and watch the messages stream.

(3) Wire a push-button between digital pin **D2** and **3V3**, with a **10 kΩ pull-down** resistor from D2 to GND. Wire an external **LED** on **D3** with a **220 Ω** series resistor to GND. Write a sketch that reads D2 every loop iteration (polling), toggles the D3 LED on each *fresh* (debounced) button press, and logs each press to Serial with a timestamp.

(4) Rewrite (3) using an attached interrupt handler (`attachInterrupt()`) on the *rising* edge of D2. The ISR should set a `volatile bool` flag that the main loop consumes. Add one Serial line per detected press that reports the **number of microseconds elapsed in** `loop()` (use `micros()`) since the previous button event.

(5) Open the `starter/hello_world/hello_world.ino` sketch provided in the lab repo (the canonical TFLM sine-wave demo, Apache-2.0, mirrored from Google's `tflite-micro-arduino-examples`). Read it top to bottom (~100 lines) and write one-sentence comments in your lab report for: the **model array**, the `AllocateTensors()` call, the **inference loop** (`Invoke()`), and the **output post-processing**.

(6) Install the **Arduino_TensorFlowLite** library from the `Arduino_TensorFlowLite.zip` in this lab folder via **Sketch → Include Library → Add .ZIP Library…**. Compile and flash the hello_world sketch (the first compile takes 1–2 minutes). Verify three independent indicators that the sketch is running correctly: (a) the Nano's orange on-board LED fades **sinusoidally** with a ~10 s period; (b) the Serial Monitor at 9600 baud prints integers cycling smoothly between 0 and 255; (c) the Serial Plotter at 9600 baud draws a clean sine wave between 0 and 255.

(7) Starting from the `starter/b3-hello-world-instrumented/` skeleton (a copy of the working hello_world sketch with two `// TODO(student) B.3:` blocks marking where the timing goes), instrument it to measure **inference latency**: wrap each `Invoke()` call with `micros()` before and after and report the **min, mean, and max** over at least 100 successive inferences. Record **flash usage** and **RAM usage** from the Arduino IDE compile output (printed after a successful compile). **Print the latency summary line with `Serial.print()` / `Serial.println()`, not `MicroPrintf`** — the TFLM v2.4.0-ALPHA `MicroPrintf` bundled with this lab does not accept the `l` length modifier, so `%lu` for the `unsigned long` that `micros()` returns prints garbage. Also note that `micros()` on the nRF52840 has ~4 µs resolution; the mean over the window is the meaningful number (the min may collapse to a single quantum across many samples).

(8) Write two short reflection paragraphs (3–5 sentences each) in your lab report: (a) Given your measured flash and RAM consumption, roughly what fraction of the Nano's 1 MB flash and 256 KB SRAM is the model + TFLM runtime using? Could you realistically fit a model that is 10×, 50×, or 100× larger? (b) The hello-world model is a *regression* model. Describe one embedded AI application from Lecture 1 that would also be a regression problem, and one that would be a classification problem.

Note: (1) You will need to start with a brief *model* of what the system should do — for the GPIO parts, a simple state diagram of "idle → press detected → LED toggled" suffices; for the TFLM part, a block diagram of the *sense* → *infer* → *act* loop (model output y → PWM brightness → LED). (2) You are required to use the *Arduino C/C++* programming model (setup() / loop(), libraries via the IDE). A reference wiring diagram for (3) and (4), and skeleton starter sketches for (3), (4), and (7) — file structure plus `// TODO(student)` hints — are provided in the lab repository at https://github.com/ACANETS/eece-4862-5862-labs under the `lab-1/starter/` folder. The skeletons compile and upload as-is so you can confirm your toolchain, but credit comes from your implemented logic: a starter submitted with its TODOs unfilled earns no credit for that part. `starter/hello_world/` is the one complete sketch — you simply run it for (5) and (6).

Additional requirements for EECE.5862 students

5862 students must complete **one** of the three options below (4862 students may attempt one for up to +2 bonus credit, capped at the lab's 20-point maximum).

(9) **Option C.1 — Polling vs. interrupt latency characterization.** Compare your polling (3) and interrupt (4) sketches quantitatively. Trigger 50 button presses at three different main-loop workloads (insert `delay(1)`, `delay(10)`, and `delay(50)` in the main loop). For each workload, report the **worst-case response time** from press to LED toggle. Write two short paragraphs analyzing the relationship between main-loop workload and worst-case response time in each style.

(10) **Option C.2 — IMU-driven LED blink rate.** Install the `Arduino_BMI270_BMM150` library and use the on-board BMI270 IMU to read the magnitude of the accelerometer vector each loop iteration. Map that magnitude to an LED blink rate (slower when the board is still, faster when moving). Include a short serial log of the acceleration magnitudes and corresponding blink intervals.

(11) **Option C.3 — Latency under three optimization levels.** Re-compile the hello_world sketch (with the latency instrumentation from (7)) under three optimization settings (`-O0`, `-O2`, `-O3`) and report inference latency (mean over 100 invocations) and final flash size for each level. Two short paragraphs of analysis: what does each level cost, and what changes at runtime? Concrete steps: (i) Locate the Mbed Nano core directory: macOS `~/Library/Arduino15/packages/arduino/hardware/mbed_nano/<version>/`, Linux `~/.arduino15/packages/arduino/hardware/mbed_nano/<version>/`, Windows `%LOCALAPPDATA%\Arduino15\packages\arduino\hardware\mbed_nano\<version>\` — it contains `platform.txt`. **Do not edit `platform.txt`** (the IDE overwrites it on core updates). (ii) Create a sibling file named **`platform.local.txt`** with the single line `compiler.optimization_flags=-O3`. (iii) Fully quit and restart the Arduino IDE, then enable **File → Preferences → "Show verbose output during: compile"** and verify you see `-O3` in the gcc command lines (the core's default is `-Os`). (iv) Record mean latency over 100 inferences and the flash size; then change the one line to `-O2`, rebuild, record; then `-O0`, rebuild, record. (No further IDE restart needed after the first.) Sanity check: all three numbers should differ from each other and from the IDE default — if two levels match, the override didn't take effect. *(Alternative: pass `--build-property "compiler.optimization_flags=-O3"` to the Arduino CLI if you prefer the command line.)*

Hints:

Skeleton sketches with `// TODO(student)` markers are provided for (3), (4), and (7) under `lab-1/starter/` — build one unmodified first to confirm your toolchain, then fill the TODOs. A reference wiring diagram for (3) and (4) is included in the lab repository under the `lab-1/figures/` directory. The `Arduino_TensorFlowLite.zip` in the lab folder is the **v2.4.0-ALPHA** runtime mirrored from the *TinyML Cookbook 2E* companion repo; the file should be **~1.48 MB** (if your download is ~219 KB, you grabbed GitHub's HTML preview by mistake — re-download from the *raw* link). The first TFLM compile takes 1–2 minutes because the library is compiled from source; subsequent builds are cached.

Lab Materials:

Required Parts:

Part name	Quantity	Notes
Arduino Nano 33 BLE Sense Rev2 (SKU ABX00070)	1	with headers
Solderless breadboard (830 tie-points)	1	
5 mm LED	1	external (on-board LED also used in (1), (2), (6))
220 Ω resistor	1	LED current limiter

Part name	Quantity	Notes
10 kΩ resistor	1	push-button pull-down
Tactile push-button	1	breadboard-mountable
USB-A to micro-USB cable	1	must be **data**-capable, not power-only
Jumper wires (M-M, M-F)	as needed	

Required Software:

(a) Arduino IDE 2.x: https://www.arduino.cc/en/software

(b) Arduino Mbed OS Nano Boards package — install via the IDE's Boards Manager; provides the Nano 33 BLE Sense target.

(c) Arduino_TensorFlowLite library (TFLM runtime) — install via **Sketch** → **Include Library** → **Add .ZIP Library…** using `lab-1/Arduino_TensorFlowLite.zip` from the lab repo.

Lab Instructions

Step 1:

Install the Arduino IDE 2.x and the Mbed OS Nano Boards package. Plug the Nano into your laptop and complete requirement (1) — Blink. This verifies the toolchain, the board target, and the USB cable end-to-end before you wire anything up.

Step 2:

Modify Blink to log to Serial (requirement (2)). Open the Serial Monitor at 9600 baud and confirm you see one message per LED state change.

Step 3:

Wire your breadboard as described in requirement (3): push-button on D2 with a 10 kΩ pull-down, external LED on D3 with a 220 Ω current-limiter. Refer to the wiring diagram in the lab repo (`lab-1/figures/lab1-button-led-wiring.png`). Double-check polarities and continuity before applying power.

Step 4:

Implement and test the polling sketch (requirement (3)) and the interrupt sketch (requirement (4)) on the wiring from Step 3. Verify in the Serial Monitor that each *fresh* press produces exactly one log line (no bouncing).

Step 5:

Install the `Arduino_TensorFlowLite` library from the `.zip` in the lab folder. Open the `hello_world.ino` sketch in the `starter/hello_world/` folder, read it through (requirement (5)), then compile and upload (requirement (6)). Verify all three indicators (LED fade, Serial Monitor stream, Serial Plotter sine wave).

Step 6:

Add the latency instrumentation to the hello_world sketch (requirement (7)). Capture flash usage, RAM usage, and the min/mean/max inference latency over ≥100 invocations.

EECE5862:

<For 5862 students>: Complete one of options C.1, C.2, or C.3 from requirements (9)–(11). Document your method, results, and analysis paragraphs in the lab report.

Step 7:

Test and debug your design. Write your lab report following the **Lab Report Template** posted on Canvas. Include all required deliverables: photos of the breadboard wiring (Steps 3–4), screenshots (Serial Monitor / Serial Plotter), serial logs, code listings (screenshots of key snippets — do not paste 1000-line dumps), the latency / resource table, and your reflection paragraphs. Per the syllabus **AI Use Policy**, disclose any AI tool you used (tool, prompts, output, and modifications).

Step 8:

Push your source code to a **private** GitHub repository. Submit the lab report PDF and a `.zip` of your Arduino sketches via Canvas.

Step 9:

Record a short demo video showing your working setup (push-button toggling the D3 LED, and the hello_world sketch fading the on-board LED in sync with the Serial Plotter sine wave) and post it on YouTube. Include the video URL in your lab report.

###

Please refer to lab demonstration and report guidelines for demonstration and report writing.